

Python for Data Engineering

Document ID	DOC009
Project	Python
Category	Programming
Batch	batch_3

Purpose: Original educational notes created as a searchable PDF corpus for a portfolio-scale incremental RAG pipeline. The material is designed to contain meaningful technical sections that naturally produce multiple retrieval chunks.

Python in Data Engineering

Python is widely used in data engineering for file processing, API integration, orchestration scripts, data validation, automation, and distributed processing through libraries such as PySpark. Clear functions, predictable data structures, logging, configuration management, and error handling are generally more valuable than complicated object-oriented designs for small pipeline utilities.

Practical note. Readable pipeline code favors explicit configuration, small reusable functions, safe secret handling, and enough logging to understand what a run actually did.

Files and Paths

The `pathlib` module provides object-oriented path handling and works well for pipeline code. A program can iterate through directories, filter by suffix, create output folders, and construct platform-independent paths. File-processing pipelines should avoid assuming that directory iteration order represents business order unless that behavior is explicitly controlled.

Practical note. Readable pipeline code favors explicit configuration, small reusable functions, safe secret handling, and enough logging to understand what a run actually did.

Functions and Modules

Functions make pipeline stages reusable and testable. A function should have a clear responsibility, inputs, and outputs. Related functions can be grouped into a small number of modules instead of creating a separate file for every operation. A main entry point can coordinate extraction, cleaning, chunking, embedding, storage, and audit logging.

Practical note. Readable pipeline code favors explicit configuration, small reusable functions, safe secret handling, and enough logging to understand what a run actually did.

Environment Variables

Secrets such as API keys should not be hard-coded in source files. A local `.env` file can hold development environment variables and should be excluded from Git. Python can load values into the process environment and validate that required configuration exists without printing the secret itself.

Practical note. Readable pipeline code favors explicit configuration, small reusable functions, safe secret handling, and enough logging to understand what a run actually did.

Hashing and Idempotency

The hashlib module can generate a deterministic digest from file bytes or normalized content. A SHA-256 hash changes when content changes, making it useful for simple duplicate or change detection. A hash is not a replacement for a business identifier, but combining document IDs with content hashes provides a practical portfolio-scale incremental ingestion mechanism.

Practical note. Readable pipeline code favors explicit configuration, small reusable functions, safe secret handling, and enough logging to understand what a run actually did.

Exceptions and Auditability

Pipeline code should distinguish successful and failed processing. Exceptions can be caught at an appropriate boundary so the audit log records failure while preserving enough error information for debugging. Catching every exception and silently continuing is dangerous because the pipeline may appear successful even though data was not processed.

Practical note. Readable pipeline code favors explicit configuration, small reusable functions, safe secret handling, and enough logging to understand what a run actually did.

End of document.